

SENIOR ENGINEER INTERVIEW PREP

Gokulakonda Gautham

GOLANG DEEP DIVE

Concurrency, Channels, Goroutines & Production Servers

20 Questions | GMP Scheduler | Concurrency Patterns | Production Architecture

Golang — Concurrency, Channels, Goroutines & Servers

Q1: What is a goroutine and how does it differ from a thread?

A goroutine is Go's fundamental unit of concurrent execution — a lightweight, cooperatively-scheduled function managed by the Go runtime, not the OS. The runtime multiplexes thousands of goroutines onto a small pool of OS threads.

Stack size: OS threads start with a fixed stack (typically 1-8MB). Goroutines start with 2KB and grow dynamically as needed (up to 1GB by default). You can run a million goroutines on a laptop.

Scheduling: Goroutines use M:N scheduling — M goroutines mapped onto N OS threads. The Go scheduler (which is work-stealing) handles context switching in user space, not the kernel. Switches happen at function calls, channel operations, syscalls, and explicit yield points.

Creation cost: Creating an OS thread: ~10-100 microseconds, MBs of memory. Creating a goroutine: ~1 microsecond, 2KB. The difference in scale is why Go services routinely spawn a goroutine per connection.

```
package main

import (
    "fmt"
    "time"
)

func main() {
    // Spawn 1 million goroutines — totally fine in Go
    for i := 0; i < 1_000_000; i++ {
        go func(id int) {
            fmt.Println("goroutine", id)
        }(i)
    }
    time.Sleep(time.Second)
}
```

Q2: What is the Go scheduler (GMP model)?

The Go runtime scheduler uses the GMP model: G (goroutines), M (OS threads/machine), P (logical processor/context). This is what enables efficient concurrent execution.

G — Goroutine: The goroutine itself: its stack, instruction pointer, and the channel/mutex it may be waiting on.

M — Machine: An OS thread. Ms execute goroutines. The runtime creates and reuses Ms. A blocked M (e.g., during a syscall) causes the runtime to spawn a new M to continue running other goroutines.

P — Processor: A logical execution context. Each P has a local run queue of goroutines. GOMAXPROCS determines how many Ps exist (default = number of CPU cores). Only goroutines assigned to a P with an M can run.

Work stealing: When a P's local queue is empty, it steals goroutines from other Ps' queues. This ensures all cores stay busy even with uneven work distribution.

```
import 'runtime'

// Control how many OS threads can execute goroutines simultaneously
runtime.GOMAXPROCS(4) // default: runtime.NumCPU()

// Yield the current goroutine, let other goroutines run
runtime.Gosched()
```

```
// See current goroutine count
fmt.Println(runtime.NumGoroutine())
```

Q3: What are channels and how do they work internally?

Channels are Go's primary mechanism for safe communication between goroutines. They implement a typed, goroutine-safe queue. The mantra: 'Do not communicate by sharing memory; share memory by communicating.'

Unbuffered channel: Both sender and receiver must be ready simultaneously. Send blocks until a receiver is ready; receive blocks until a sender sends. This is a synchronization point — a rendezvous.

Buffered channel: Has a capacity. Sends don't block until the buffer is full. Receives don't block until the buffer is empty. Decouples sender and receiver timing.

```
// Unbuffered – blocks until both sides are ready
ch := make(chan int)

go func() { ch <- 42 }() // sender goroutine
val := <-ch             // receiver – unblocks when sender sends

// Buffered – sender can send 3 values before blocking
buffered := make(chan string, 3)
buffered <- 'first'    // doesn't block
buffered <- 'second'   // doesn't block
buffered <- 'third'    // doesn't block
buffered <- 'fourth'   // BLOCKS – buffer full

// Directional channels for API clarity
func produce(out chan<- int) { out <- 1 } // send-only
func consume(in <-chan int) { v := <-in } // receive-only
```

Internal structure: Internally, a channel is a `hchan` struct with a circular buffer, send/recv queues of blocked goroutines, a mutex for concurrent access, and counts for pending elements. The runtime parks (suspends) goroutines that block on channels and wakes them when the operation becomes possible.

Closing channels: `close(ch)` signals no more values. Receivers can detect this with `v, ok := <-ch` (`ok` is `false` when closed). Range over a channel exits when closed. Never close from the receiver side; never close a nil channel (panics).

Q4: What is a WaitGroup and how does it work?

`sync.WaitGroup` is a counter for tracking goroutine completion. It solves the classic problem: 'spawn N goroutines, wait for all to finish.' The counter is thread-safe — `Add`/`Done`/`Wait` are all atomic operations.

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var wg sync.WaitGroup
    results := make([]int, 5)

    for i := 0; i < 5; i++ {
        wg.Add(1) // increment BEFORE spawning goroutine
        go func(idx int) {
            defer wg.Done() // decrement when goroutine exits

```

```

        results[idx] = idx * idx
    }(i)
}

wg.Wait() // blocks until counter reaches 0
fmt.Println(results)
}

```

Critical rule: Call Add() BEFORE launching the goroutine. If you call Add() inside the goroutine, main() might call Wait() before the goroutine has a chance to increment — race condition.

WaitGroup vs channels: Use WaitGroup when you want to wait for goroutines to finish but don't need to collect results. Use channels when you need to collect return values from goroutines. Often combined: goroutines send results to a channel; a WaitGroup closes the channel when all are done.

```

// Pattern: WaitGroup + channel for collecting results
func fanOut(inputs []string) []string {
    results := make(chan string, len(inputs))
    var wg sync.WaitGroup

    for _, input := range inputs {
        wg.Add(1)
        go func(s string) {
            defer wg.Done()
            results <- process(s)
        }(input)
    }

    go func() {
        wg.Wait()
        close(results) // safe to close: all writers done
    }()

    var out []string
    for r := range results {
        out = append(out, r)
    }
    return out
}

```

Q5: What is a Mutex and when do you use it vs channels?

sync.Mutex provides mutual exclusion — only one goroutine can hold the lock at a time. Use it when multiple goroutines need to read/write shared mutable state and you need performance over elegance.

```

type SafeCounter struct {
    mu    sync.Mutex
    count int
}

func (c *SafeCounter) Inc() {
    c.mu.Lock()
    defer c.mu.Unlock() // ALWAYS defer unlock – ensures release even on panic
    c.count++
}

func (c *SafeCounter) Value() int {
    c.mu.Lock()
    defer c.mu.Unlock()
    return c.count
}

```

RWMutex: `sync.RWMutex` distinguishes readers from writers. Multiple goroutines can hold `RLock()` simultaneously (concurrent reads). Only one goroutine can hold `Lock()` (exclusive write). Critical for read-heavy workloads like in-memory caches.

```
type Cache struct {
    mu    sync.RWMutex
    store map[string]string
}

func (c *Cache) Get(key string) (string, bool) {
    c.mu.RLock()           // multiple readers OK
    defer c.mu.RUnlock()
    v, ok := c.store[key]
    return v, ok
}

func (c *Cache) Set(key, val string) {
    c.mu.Lock()            // exclusive write
    defer c.mu.Unlock()
    c.store[key] = val
}
```

Mutex vs channels: Use Mutex for: protecting shared state, caches, counters, state machines. Use channels for: passing data between goroutines, signaling events, pipeline patterns. Rule of thumb: if you're sharing state, mutex. If you're communicating, channels.

Q6: What are common concurrency patterns in Go?

Go's concurrency primitives enable several powerful, idiomatic patterns. These are what you'll see in production Go codebases and what interviewers expect you to know.

Fan-out: Distribute work from one input channel to multiple goroutines. Each goroutine processes items independently. Useful for parallel processing of a stream.

Fan-in (merge): Collect results from multiple goroutines into a single channel. The opposite of fan-out. Often combined: fan-out to distribute, fan-in to collect results.

```
// Fan-in: merge multiple channels into one
func merge(channels ...<-chan int) <-chan int {
    out := make(chan int)
    var wg sync.WaitGroup
    wg.Add(len(channels))
    for _, ch := range channels {
        go func(c <-chan int) {
            defer wg.Done()
            for v := range c { out <- v }
        }(ch)
    }
    go func() { wg.Wait(); close(out) }()
    return out
}
```

Pipeline: Stages connected by channels. Each stage receives from upstream, transforms, sends downstream. First stage generates data; last stage consumes. Cancellation propagates via context.

Worker pool: Fixed number of goroutines consuming from a shared job channel. Controls resource usage — prevents spawning unbounded goroutines.

```
// Worker pool pattern
func workerPool(jobs <-chan Job, numWorkers int) <-chan Result {
    results := make(chan Result, numWorkers)
    var wg sync.WaitGroup
    for i := 0; i < numWorkers; i++ {
        wg.Add(1)
        go func() {
```

```

        defer wg.Done()
        for job := range jobs {
            results <- process(job)
        }
    }()
}
go func() { wg.Wait(); close(results) }()
return results
}

```

Q7: What is the select statement and how does it work?

select is like a switch for channel operations. It blocks until one of its cases can proceed, then executes that case. If multiple cases are ready simultaneously, one is chosen at random.

```

select {
case msg := <-ch1:
    fmt.Println("received from ch1:", msg)
case ch2 <- value:
    fmt.Println("sent to ch2")
case <-time.After(5 * time.Second):
    fmt.Println("timeout – no activity in 5s")
case <-ctx.Done():
    fmt.Println("context cancelled:", ctx.Err())
    return
default:
    fmt.Println("non-blocking – no case ready")
}

```

Timeout pattern: time.After() returns a channel that receives after the duration. Used in select to implement timeouts on channel operations. For production, use context.WithTimeout instead — it's cancellable and doesn't leak goroutines.

Done channel pattern: Pass a 'done' channel (or context) to goroutines. When the done channel is closed, all goroutines listening on it wake up and clean up. This is Go's idiomatic cancellation mechanism before context was added.

```

// Cancellable worker with context
func worker(ctx context.Context, jobs <-chan Job) {
    for {
        select {
        case job, ok := <-jobs:
            if !ok { return } // channel closed
            process(job)
        case <-ctx.Done():
            return // context cancelled or timed out
        }
    }
}

```

Q8: What is context.Context and why is it everywhere in Go?

context.Context is Go's standard mechanism for carrying cancellation signals, deadlines, and request-scoped values across goroutines and API boundaries. It is the idiomatic way to handle timeouts and cancellation propagation.

Why it matters: In a microservice, a single request may spawn 10 goroutines making DB calls, RPC calls, and cache lookups. If the HTTP client disconnects, context.Context lets you cancel ALL of them atomically with one cancel().

```
func handleRequest(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 5*time.Second)
    defer cancel() // ALWAYS defer cancel – prevents goroutine/resource leaks

    // All downstream calls respect this timeout
    user, err := db.GetUser(ctx, userID) // cancels if 5s exceeded
    if err != nil {
        if errors.Is(err, context.DeadlineExceeded) {
            http.Error(w, 'timeout', 504)
            return
        }
    }

    data, err := rpcClient.GetData(ctx, user.ID) // also respects timeout
}
```

Context hierarchy: Contexts form a tree. Child contexts inherit cancellation from parents. Cancelling a parent cancels all children. `context.Background()` is the root for `main()`, tests, and server init. `context.TODO()` means 'I haven't decided yet' — a code smell in production.

Values in context: `context.WithValue()` embeds request-scoped values (trace IDs, auth tokens, user ID). Use typed private keys to avoid collisions. Keep values small and read-only — context is not a general-purpose map.

Q9: How do you detect and prevent goroutine leaks?

A goroutine leak is a goroutine that is started but never exits. It consumes memory, CPU, and can hold locks or connections open. In long-running servers, leaks compound and eventually crash the process.

Common causes: Goroutine blocked on a channel receive when no one will ever send. Goroutine blocked on a channel send when no one will ever receive. Missing context cancellation. Goroutine in an infinite loop with no exit condition.

```
// LEAK: goroutine blocked forever if ctx is never cancelled
func leaky(ch <-chan int) {
    go func() {
        v := <-ch // if ch is never sent to and never closed: leak
        process(v)
    }()
}

// FIX: add context
func safe(ctx context.Context, ch <-chan int) {
    go func() {
        select {
        case v := <-ch:
            process(v)
        case <-ctx.Done():
            return // clean exit on cancellation
        }
    }()
}
```

Detection: `runtime.NumGoroutine()` — monitor this metric in production via Prometheus. `goleak` library in tests (github.com/uber-go/goleak) — asserts no goroutines leaked after each test. `pprof` goroutine profile — shows all running goroutines with their stack traces.

```
// In tests – catches goroutine leaks automatically
func TestMyFunc(t *testing.T) {
    defer goleak.VerifyNone(t)
    // ... your test code ...
}
```

Q10: How do you build an HTTP server in Go?

Go's `net/http` package provides a production-grade HTTP server out of the box. The standard library handles connection management, keep-alive, TLS, and basic routing. For complex APIs, frameworks like Gin or Chi add routing, middleware, and DI.

```
package main

import (
    "encoding/json"
    "log"
    "net/http"
    "time"
)

type Server struct {
    router *http.ServeMux
    db      *DB
}

func NewServer(db *DB) *Server {
    s := &Server{router: http.NewServeMux(), db: db}
    s.routes()
    return s
}

func (s *Server) routes() {
    s.router.HandleFunc("GET /users/{id}", s.handleGetUser)
    s.router.HandleFunc("POST /users", s.handleCreateUser)
}

func (s *Server) handleGetUser(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id")
    user, err := s.db.GetUser(r.Context(), id)
    if err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(user)
}

func main() {
    db := connectDB()
    srv := NewServer(db)
    httpServer := &http.Server{
        Addr:      ":8080",
        Handler:    srv.router,
        ReadTimeout: 10 * time.Second,
        WriteTimeout: 10 * time.Second,
        IdleTimeout: 60 * time.Second,
    }
    log.Fatal(httpServer.ListenAndServe())
}
```

Timeouts: Always set `ReadTimeout`, `WriteTimeout`, and `IdleTimeout`. Without them, slow clients can hold connections open indefinitely, exhausting file descriptors.

Q11: How do you implement middleware in Go?

Go middleware follows the Handler wrapping pattern — a function that takes an `http.Handler` and returns an `http.Handler`. This creates a chain where each middleware can execute before and after the next handler.

```
// Middleware signature
type Middleware func(http.Handler) http.Handler

// Logging middleware
func Logger(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        // Wrap ResponseWriter to capture status code
        wrapped := &responseWriter{ResponseWriter: w, status: 200}
        next.ServeHTTP(wrapped, r)
        log.Printf('%s %s %d %v', r.Method, r.URL.Path,
            wrapped.status, time.Since(start))
    })
}

// Auth middleware
func RequireAuth(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        token := r.Header.Get('Authorization')
        if !validateToken(token) {
            http.Error(w, 'Unauthorized', 401)
            return
        }
        next.ServeHTTP(w, r)
    })
}

// Chain middlewares
handler := Logger(RequireAuth(rateLimiter(mux)))
```

Q12: What is an interface in Go and how does it differ from other languages?

Go interfaces are implicit — a type satisfies an interface by simply implementing its methods. There is no 'implements' keyword. This is structural typing, not nominal typing.

```
type Stringer interface {
    String() string
}

type User struct { Name string }

// User satisfies Stringer implicitly – no declaration needed
func (u User) String() string {
    return 'User: ' + u.Name
}

// Any type with a String() method can be passed here
func print(s Stringer) {
    fmt.Println(s.String())
}
```

Interface internals: An interface value has two components: a type pointer and a data pointer. This is why a non-nil interface can hold a nil pointer — the interface itself is non-nil because the type is set, even if the value is nil. This is a common Go gotcha.

Empty interface: `interface{}` (or 'any' in Go 1.18+) accepts any type. Used for generic containers before generics. Prefer typed interfaces or generics over `interface{}` — you lose type safety.

Interface design: Keep interfaces small. The Go standard library's iconic example: `io.Reader` has one method. `io.Writer` has one method. They compose: `io.ReadWriter = io.Reader + io.Writer`. Accept interfaces, return concrete types.

Q13: What are Go generics and when do you use them?

Go 1.18 introduced generics via type parameters. They let you write functions and data structures that work with multiple types without sacrificing type safety or resorting to interface{}.

```
// Generic function – works for any ordered type
func Min[T constraints.Ordered](a, b T) T {
    if a < b { return a }
    return b
}

Min(3, 5)           // int
Min(3.14, 2.71)     // float64
Min("apple", "banana") // string

// Generic data structure
type Stack[T any] struct {
    items []T
}

func (s *Stack[T]) Push(v T) { s.items = append(s.items, v) }
func (s *Stack[T]) Pop() T    { n := len(s.items)-1; v := s.items[n]; s.items = s.items[:n]; return v }
```

When to use: Generic utility functions (Map, Filter, Reduce over slices), generic data structures (Stack, Queue, Set), functions that work on multiple numeric types. Don't over-apply — if an interface works cleanly, prefer it.

Q14: How does error handling work in Go? Why no exceptions?

Go treats errors as values, not exceptions. Functions return an error as the last return value. Callers must explicitly handle or propagate it. This makes error paths visible and forces engineers to think about failure.

```
// Sentinel errors – for comparison
var ErrNotFound = errors.New('not found')

// Custom error type with context
type DBError struct {
    Query string
    Err   error
}

func (e *DBError) Error() string { return fmt.Sprintf('query %q failed: %v', e.Query, e.Err) }
func (e *DBError) Unwrap() error { return e.Err }

// Wrapping errors (Go 1.13+)
if err := db.Query(q); err != nil {
    return fmt.Errorf('GetUser %d: %w', id, err) // %w wraps for Unwrap
}

// Checking wrapped errors
if errors.Is(err, ErrNotFound) { ... } // checks chain
var dbErr *DBError
if errors.As(err, &dbErr) { ... }     // extracts type from chain
```

Why not exceptions?: Exceptions create invisible control flow. In Go, every function that can fail says so in its signature. You cannot accidentally ignore an error — the compiler enforces that you at least acknowledge it (even if you `_ it`, which is intentional).

panic and recover: panic is Go's exception for truly unrecoverable errors (nil dereference, out-of-bounds). `recover()` in a deferred function catches panics. HTTP servers use this to prevent one panicking handler from crashing the entire server.

Q15: What is the Go memory model and how do goroutines communicate safely?

The Go memory model defines when writes in one goroutine are guaranteed to be visible to another. Without synchronization, the compiler and CPU can reorder operations, making concurrent code behave unexpectedly.

Happens-before: The key concept. If event A happens-before event B, then B is guaranteed to see A's effects. Go channels, sync primitives, and goroutine creation/exit all establish happens-before relationships.

```
var x int

// UNSAFE: no synchronization – goroutine may not see x = 1
go func() { x = 1 }()
fmt.Println(x) // might print 0

// SAFE via channel – channel send happens-before receive
done := make(chan struct{})
go func() {
    x = 1
    done <- struct{}{} // send establishes happens-before
}()
<-done
fmt.Println(x) // guaranteed to print 1

// SAFE via sync.Once – guaranteed to run exactly once
var once sync.Once
once.Do(func() { x = 1 }) // safe for initialization
```

Race detector: Always run tests with `-race` flag. `go test -race ./...` uses the ThreadSanitizer to detect concurrent reads/writes without synchronization. Find races in CI, not production.

Q16: How do you structure a production Go microservice?

Go services typically follow a clean layered architecture with dependency injection through constructors. The key principle: components depend on interfaces, not concrete types, making testing straightforward.

```
// Project structure
myservice/
├── cmd/
│   └── server/
│       └── main.go // entry point, wires dependencies
├── internal/
│   ├── handler/ // HTTP handlers
│   ├── service/ // business logic
│   ├── repository/ // data access layer
│   └── middleware/ // auth, logging, rate limit
├── pkg/ // exportable packages
├── migrations/ // DB migrations
└── config/ // config structs + loading

// main.go – wires everything together
func main() {
```

```

cfg := config.Load()
db := database.Connect(cfg.DSN)

// Dependency injection via constructors
repo := repository.NewUserRepo(db)
svc := service.NewUserService(repo)
handler := handler.NewUserHandler(svc)

srv := &http.Server{
    Addr:    cfg.Port,
    Handler: buildRouter(handler),
}

// Graceful shutdown
go func() { srv.ListenAndServe() }()
quit := make(chan os.Signal, 1)
signal.Notify(quit, os.Interrupt, syscall.SIGTERM)
<-quit

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()
srv.Shutdown(ctx)
}

```

Graceful shutdown: On SIGTERM, stop accepting new connections, wait for in-flight requests to finish (with a timeout), then close DB connections and other resources. Kubernetes sends SIGTERM before killing the pod — you have ~30 seconds.

Q17: What is sync.Once, sync.Pool, and sync.Map — when do you use each?

sync.Once: Guarantees a function runs exactly once, even under concurrent access. Used for lazy initialization of expensive resources (DB connections, config loading, singleton patterns).

sync.Pool: A pool of temporary objects that can be reused across goroutines. Reduces GC pressure for frequently allocated/deallocated objects. The GC may reclaim pool objects — don't use for state that must persist. Common use: byte buffers, JSON encoders.

sync.Map: A concurrent map that is optimized for two specific access patterns: (1) keys are written once but read many times, (2) goroutines access disjoint key sets. For general concurrent map access, a regular map + RWMutex is often clearer and faster.

```

// sync.Once – lazy init
var (
    instance *DB
    once     sync.Once
)
func GetDB() *DB {
    once.Do(func() { instance = connectDB() })
    return instance
}

// sync.Pool – reuse allocations
var bufPool = sync.Pool{
    New: func() interface{} { return new(bytes.Buffer) },
}
func handler(w http.ResponseWriter, r *http.Request) {
    buf := bufPool.Get().(*bytes.Buffer)
    buf.Reset()
    defer bufPool.Put(buf)
    // use buf ...
}

```

Q18: What is defer and how does it work in Go?

defer schedules a function call to run when the surrounding function returns, regardless of how it returns (normal return, panic, or runtime error). Multiple defers in one function execute in LIFO order.

```
func processFile(path string) error {
    f, err := os.Open(path)
    if err != nil { return err }
    defer f.Close()    // guaranteed to run on any return path

    mu.Lock()
    defer mu.Unlock()  // guaranteed unlock – even on panic

    return processData(f)
}

// LIFO order – defers stack up
func example() {
    defer fmt.Println("third")    // runs last
    defer fmt.Println("second")
    defer fmt.Println("first")   // runs first
}
// Output: first → second → third
```

Arguments evaluated immediately: The arguments to a deferred function are evaluated when the defer statement is reached, not when the deferred function actually executes. This surprises people with loop variables.

Named return values: A deferred function can modify named return values. This is used to transparently add error wrapping: ``defer func() { if err != nil { err = fmt.Errorf("layer: %w", err) } }()``
